

There are actually very few data types available in SQLite: *NULL*, *INTEGER*, *REAL*, *TEXT* and *BLOB*. Significantly, there is no data type for date and time values. Dates and times can be stored in the international format as *TEXT*, a *UNIX* timestamp as *INTEGER*, or as a number of Julian days as *REAL*. To work with dates and times, SQLite provides a few functions, which you can read about in the documentation.

Also of note is how SQLite handles sizes or ranges of data values. SQLite only has fixed ranges for data values. For example, even if you specify a number of characters for *VARCHAR*, the maximum number of characters allowed is actually the maximum allowed by *TEXT*. The *INTEGER* type allows a maximum of 8-byte integers, and the *REAL* type is the IEEE 8-byte floating-point number type.

The strangest part of SQLite, however, is that you can specify literally anything for a column's data type! For example, you can use the following *CREATE TABLE* statement:

```
CREATE TABLE strange_table (x strange_type);
```

This will succeed because SQLite uses a fixed set of rules to determine the affinity of a column, based on sub-strings of what is specified for the column's type. The affinity of a column is the hint for the type of data to be stored in the column, specified as the column's 'data type' in the *CREATE TABLE* statement. If the type name has 'int' in it, the affinity will be *INTEGER*. For example, *INT*, *INTEGER*, *MY_INTEGER_TYPE*, and *FLOATING POINT* are all *INTEGER*!

Similarly, type names in which 'char', 'clob' or 'text' occur as a sub-string are treated as *TEXT*. If no type name is specified, or if the sub-string 'blob' occurs in the type name, the special affinity *NONE* is used. The sub-strings 'real', 'float' and 'double' create the *REAL* affinity. All other type names cause the column to have *NUMERIC* affinity, which can be used to store both integral and floating-point values. For example, the type name *STRING* will create the *NUMERIC* affinity. These rules are applied in this order, which is the reason that *FLOATING POINT* has *INTEGER* affinity, not *REAL*.

A final quirk to be aware of is that SQLite stores a unique row ID for every row in every table, and any column declared with a 'type' of *INTEGER PRIMARY KEY* becomes an alias for the row ID. This requires the exact name '*INTEGER*', and not a sub-string like '*INT*'. You may use the *AUTOINCREMENT* parameter for an *INTEGER PRIMARY KEY*, but not any other. Using other constraints, such as *ASC* or *DESC*, requires splitting the constraint into a separate constraint section, like *PRIMARY KEY(<column_name> <other constraints>)*. If you do not use an *INTEGER PRIMARY KEY* column, you can retrieve the row ID using the column name *ROWID*, *OID* or *_ROWID_*.

All of this is very likely to surprise users familiar with other, more traditional SQL-based database management systems, but it does not cause problems in the intended use cases of SQLite. Besides, the usual data types allowed by SQL are covered correctly under the rules. You just need to decide on a type for date and time values.

An example

We will now consider a small example of how SQLite can be used with PHP via the PDO library, which is the recommended method of using SQLite 3 with PHP. We will create a simple version of the traditional *products* table used to illustrate many database concepts. Only a single table will be used to illustrate working with SQLite. To be able to run the example successfully, ensure that you have enabled proper privileges (see the warning above).

The database

To run the text-mode program for SQLite 3, named *sqlite3*, open a terminal. Since the SQLite database will be stored in a single portable file, either change the working directory to the one in which you want to create the file, and then run *sqlite3* with just the file-name as an argument; or you can run *sqlite3* and specify the path to the file as the argument. For example, *sqlite3 Projects/products.sqlite* or *sqlite3 Projects/products.db*. The extension you use for the file name is immaterial, but use the name *products* for this example, or substitute it everywhere with any other name you choose.

The SQLite program should now be awaiting input at the *sqlite>* prompt. You can enter *.help* to see the meta-commands that can be used with *sqlite3*, or enter SQL statements terminated by the semicolon. (Note the dot '.' in *.help*. All meta-commands begin with a dot.)

Create the *products* table as follows:

```
CREATE TABLE products (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT,
  price NUMERIC
);
```

Statements can be split across multiple lines. The prompt will accept the input as part of the statement until a semicolon is entered. You can use the *.schema* meta-command to look at the structure of tables. For example, *.schema products*.

Enter a few records after this, then issue a *SELECT* * query on *products*. The output may be in the form of values separated by a bar symbol (|). You can change the separator symbol to something else, or you can change the output to a more traditional form. There are several modes of output. To get the traditional format, issue the meta-command *.mode column*. This may still not display the